Práctica 2: Revolución

Informática Gráfica

1. Introducción:

El proyecto entregado como práctica 2 parte de la base del trabajo que se realizó en la **anterior práctica**, manteniendo todos sus nodos, script y configuraciones en cuanto a la fuente de luz, la cámara etc. Para mantener el espacio limpio y se viese el trabajo realizado en esta nueva sesión, las figuras entregadas en la primera parte como el cubo, la pirámide original y el prisma se mantienen dentro de la escena pero desplazadas por defecto.

De igual manera por defecto al cargar el proyecto, los objetos que se han importado como se especificaba en el guión **no se encuentran presentes en la escena**, permanecen ocultos para no rebosar. Se ha priorizado que las figuras como el jarrón, el cono y la pirámide con sombreado suave que contienen programación en los scripts aparezcan al inicio.

Para ver las figuras cargadas **podemos hacerlas visibles** y quitar las otras figuras para que no se solapen.

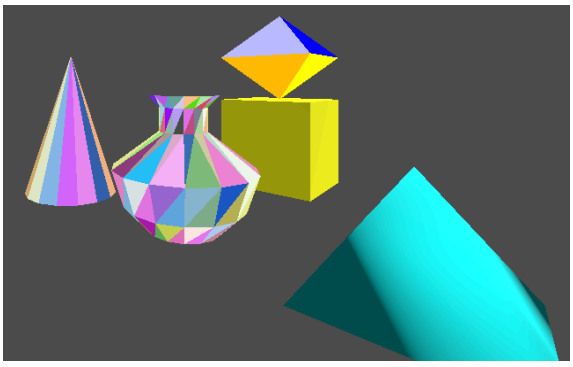
2. Controles e interacciones:

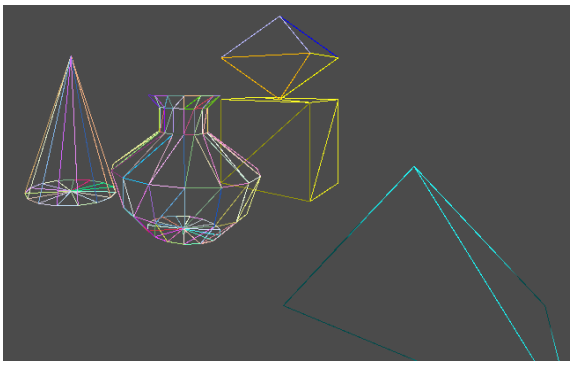
Para poder interactuar con la escena después de ejecutarla, hay una serie de **controles por teclado** que podemos ejecutar. Algunos vienen heredados de la práctica anterior y otros son nuevos de esta. Estos controles son:

1. **Rotar la cámara:** Las teclas de dirección de izquierda y derecha nos permiten movernos y orbitar a través de la escena.
2. **Visualizar las aristas (W):** Al pulsar la tecla “W”, se activa o desactiva el modo alambre utilizando el *“Rendering Server”.* Esto lo que hace es obligar a Godot a dibujar únicamente las aristas de todos los objetos presentes en pantalla, el cual ha sido muy útil para visualizar las aristas de los modelos descargados o bien las uniones de los puntos del perfil a la hora de realizar la revolución.

3. **Modos de visualización de mallas:** Tenemos distintos modos con los que visualizar los distintos modelos de la escena, podemos ir alternando entre ellos con los números 1,2 3 y 4.
    a. Con **el 1** se muestra el sólido sin iluminación aplicando un color distinto a cada cara.
    b. Con **el 2** tenemos iluminación plana, las caras que no se les ve la luz se ven oscuras.
    c. Con **el 3** se muestran las aristas pero de forma "local", esto se hace dibujando la figura con *PRIMITIVE_LINES* en vez de *triangles*.
    d. Con **el 4** se aplica el sombreado suave utilizando las normales en los vértices para que la luz pase por la superficie sin marcar los bordes en los triángulos que forman los modelos.

A continuación se muestran dos imágenes, una con el modo 1 y otra con la tecla W activa para mostrar las aristas de las figuras:





3. *Nuevas figuras y modelos cargados*

En esta práctica se han añadido nuevas figuras además de cargar dos modelos distintos de assets externos descargados.

En cuanto a **modelos importados** se han añadido dos distintos para comprobar como importar estos a través de la interfaz de godot. Los objetos importados son:

1. Un modelo en formato **.glb** con forma de Pistola a través del siguiente enlace: https://poly.pizza/m/E7IaG9TptR
2. Un modelo en formato **.obj** con su respectivo archivo **.mtl** para los materiales de una Katana a través del siguiente enlace: https://poly.pizza/m/7Ly4_Ou8HXK

A continuación se muestra una imagen de ambos modelos en pantalla, una vez que hemos hecho estos visibles y hemos puesto invisibles el resto:



En cuanto a **figuras generadas por código** se ha programados 3 figuras nuevas:

1. **Pirámide con el sombreado suave:** Fue la primera en crear. En esta creamos inicialmente la lista de vértices, generamos las normales automáticamente con la función *"generate_normals()"*. Esto hace que godot simule superficies curvas en las aristas y se note menos cuadricular los polígonos.
2. **Jarrón y Cono por revolución:** Para ello hemos creado una función que a partir de un perfil y otros parámetros (esta función la explicaremos en el siguiente apartado) genera una figura haciendo revolucionar estas aristas insertadas en el perfil.

4. Función de revolución

Para **reutilizar código y no repetir operaciones** en el propio script de cada modelo revolucionado, dentro archivo de nuestro proyecto "*modos_malla.gd*". Su objetivo es tomar el dibujo 2d del perfil y hacerlo girar para construir un objeto 3d sólido. Para entender el funcionamiento de esta función primero analizaremos su cabecera y los datos que esta maneja:

***"static func generar_revolucion**(perfil: Array, lados: int, tapa_inf: bool, tapa_sup: bool, vertices_out: Array, caras_out: Array):"*

Recibe como entradas el **perfil**, que es una lista con las coordenadas que dibujan la silueta, el número de **lados** que queremos que tenga la figura circular y los **dos booleanos** para las tapas inferior y superior que decide si la figura se genera tapada por sus extremos o no.

Además obtenemos como salidas **los vértices y triángulos generados** para que el script de la propia figura se encargue de llamar a la propia función ya obtenga de forma limpia los datos necesarios para meterlos en la malla.

El algoritmo interno se divide en 3 fases principales:

1. **Generamos los vértices por rotación:**

```
# Creamos los vertices nuevos rotando el perfil
for i in lados:
    # TAU es una variable equivalente a 2*PI, es decir 360 grados en rads
    var angulo = i*(TAU/lados)

    for p in perfil:
        # Rotamos cada punto sobre el eje Y
        var p_rotado = p.rotated(Vector3.UP, angulo)
        vertices_out.append(p_rotado)
```

Utilizamos un bucle que divide los 360 grados en el número de lados insertado por parámetro, en cada paso se coge el perfil original y se rota alrededor del del eje Y. Todos esos puntos que hemos generado los vamos guardando dentro de *vertices_out*.

2. **Obtenemos los triángulos:**

```
# Conectamos los vertices que acabamos de crear para generar las caras
for i in lados:
    var sig_i = (i+1)%lados # EL %lados es necesario para que el ultimo se conecte con el primero xd

    for j in num_puntos - 1:
        var abajo_izq = i*num_puntos + j
        var arriba_izq = i*num_puntos + (j + 1)
        var abajo_der = sig_i*num_puntos + j
        var arriba_der = sig_i * num_puntos +(j + 1)

        # Dividimos el cuadrado en 2 triángulos
        caras_out.append([abajo_izq, arriba_izq, arriba_der])
        caras_out.append([abajo_izq, arriba_der, abajo_der])
```

Recorremos los lados que hemos generado, uniendo estos entre cada uno con el contiguo. Sin olvidarnos de conectar el último con el primero. Lo que vamos haciendo

es identificar grupos de 4 vértices formando un cuadrado, que después dividiremos en 2 triángulos en sentido horario para que las normales apuntes hacia el exterior.

3. **Cerramos las tapas de la figura:**

Si se indica mediante los booleanos que necesitamos cerrar o bien la tapa superior o la inferior, en el código calculamos el centro exacto de la base tomando la altura y el primer punto (o último) del perfil. Este vértice central se añade a la lista y se conecta mediante un bucle con todos los vértices, generando triángulos que van cerrando la malla.

5. Uso de Inteligencia Artificial

Durante el desarrollo de esta práctica se ha utilizado como asistencia de apoyo la IA de forma puntual y como herramienta de aprendizaje en los siguientes puntos:

1. Para el **funcionamiento interno de clases** específicas de Godot como el Shading Per Pixel o el Shading per Vertex, a nivel de documentación y pregunta de dudas sobre estas
2. Como ayuda a la **depuración,** para identificar errores sintácticos y lógicos que no terminaba de identificar de forma manual dentro del código.
3. Para **diseñar los puntos iniciales del jarrón,** al hacerlo de forma manual con sus coordenadas a mano no quedaba nada bien, para que el resultado se viese más estético genere estos puntos con IA.
4. En el **fragmento de código** con respecto a la generación de las tapas superiores e inferiores en la función de revolución, me costó bastante plantear inicialmente la idea de generar el bucle para conectar el punto central con el anillo exterior de vértices.